Capacitação em Linguagem de Programação C

Aula 3 – Novos Tipos de Dados com struct, typedef, union e enum - parte 1 (Nível Avançado)

Roteiro

- Criando novos tipos de dados com struct
- Novos nomes para os tipos existentes
- Usando typedef com struct
- Inicializando estruturas
- Atribuições entre estruturas
- Estruturas aninhadas
- Passando estruturas para funções
- Funções que retornam uma estrutura
- Matrizes de estruturas
- Estruturas com campos bit

Introdução

- Talvez você já tenha se deparado com um problema de programação no qual deseje agrupar sob um único nome um conjunto de tipos de dados não similares.
- Seu primeiro impulso seria, provavelmente, usar uma matriz.

Introdução

- Como matrizes requerem que todos os seus elementos sejam do mesmo tipo, você, certamente, forçaria a resolução do problema selecionando uma matriz para cada tipo de dado, mas o resultado tornaria o programa ineficiente na maneira de manejar os dados.
- O problema de agrupar dados de variáveis que agrupam dados geralmente desiguais; ao passo que matrizes agrupam dados similares.

Introdução

- Os itens de dados de estrutura são chamados de membros, e os da matriz, de elementos.
- Em algumas linguagens de programação, estruturas são chamadas de registros.
- O exemplo tradicional de uma estrutura é o registo de uma folha de pagamento:
  - Um funcionário é descrito por um conjunto de atributos, tais como nome (string), o número do seu departamento (inteiro), salário (float), e assim por diante.

Introdução

- Possivelmente, haverá outros funcionários, e você vai querer que seu programa os guarde, formando uma matriz de estruturas.
- Nesta aula, exploraremos a criação de novos tipos de dados por meio de estruturas e uniões.
- Você já conhece os cinco tipos de dados simples que estão predefinidos no compilador (char, int, float, double e void), os quais representam um único item de informação.

Introdução

- Com base nesses tipos, podemos definir tipos complexos, que possibilitem agrupar um conjunto de variáveis de tipos diferentes som um único nome.

Roteiro

- Criando novos tipos de dados com struct
- Novos nomes para os tipos existentes
- Usando typedef com struct
- Inicializando estruturas
- Atribuições entre estruturas
- Estruturas aninhadas
- Passando estruturas para funções
- Funções que retornam uma estrutura
- Matrizes de estruturas
- Estruturas com campos bit

Criando Novos Tipos de Dados com struct

- Por meio da palavra-chave struct definimos um novo tipo de dado.
- Definir um tipo de dado significa informar ao compilador seu nome, tamanho em bytes e a forma como deve ser armazenado e recuperado na memória.
- Após ter sido definido, o novo tipo existe e pode ser utilizado para criar variáveis de modo similar a qualquer tipo simples.

Criando Novos Tipos de Dados com struct

- O exemplo a seguir cria um tipo de dado que pode armazenar as informações de um aluno da escola.
- Observe a listagem:

Criando Novos Tipos de Dados com struct

Prog1.c

```c
/* Prog1.c */
/* Cria uma estrutura para armazenar dados de um aluno */
#include <stdio.h>
#include <stdlib.h>

struct Aluno          /*Inicio da definição da estrutura */
{
    int nmat;         /* Número da matrícula */
    float nota[3];    /* Notas */
    float media;      /* Média */
};                 /* Fim da definição */

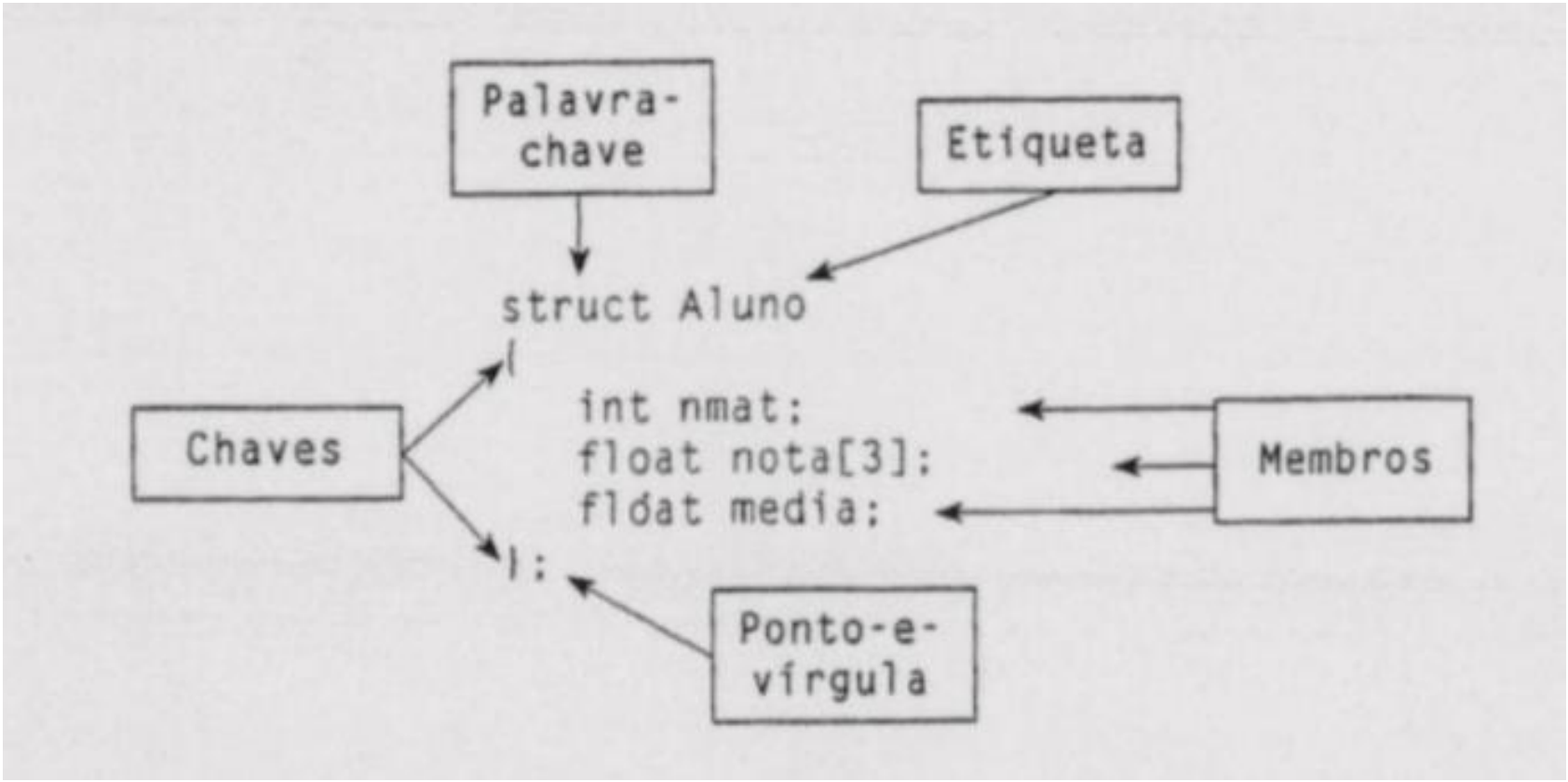
```

Criando Novos Tipos de Dados com struct

```
13  int main()
14  {
15      struct Aluno Jose; /* Declara uma variável do tipo struct Aluno */
16
17      Jose.nmat   = 456;
18      Jose.nota[0]= 7.5;
19      Jose.nota[1]= 5.2;
20      Jose.nota[2]= 8.4;
21      Jose.media  = (Jose.nota[0] + Jose.nota[1] + Jose.nota[2])/3.0;
22
23      printf("Matrícula: %d\n", Jose.nmat);
24      printf("Média    : %.2f\n", Jose.media);
25
26      system("PAUSE");
27      return 0;
28  }
```

Definindo a Estrutura

- A definição da estrutura informa como ela é organizada e quais são seus membros.



Definindo a Estrutura

- A definição de uma estrutura não cria nenhuma variável, somente informa ao compilador as características de um novo tipo de dado.
- Não há nenhuma reserva de memória.
- A palavra struct indica que um novo tipo de dado esta sendo definido e a palavra Aluno será sua etiqueta.
- O nome do nosso novo tipo de dados é struct Aluno.

Definindo a Estrutura

- No nosso exemplo, definimos o tipo struct Aluno antes de main(), o que permite um acesso global a todas as funções definidas no programa.
- Poderíamos colocar essa definição dentro de uma função, restringindo o acesso às instruções do mesmo bloco e escritas abaixo dela.

Definindo a Estrutura

Prog2.c

```c
/* Prog2.c */
/* Cria uma estrutura para armazenar dados de um aluno */
#include <stdio.h>
#include <stdlib.h>

int main()
{
    struct Aluno        /*Inicio da definição da estrutura */
    {
        int nmat;       /* Número da matrícula */
        float nota[3];  /* Notas */
        float media;    /* Média */
    };                    /* Fim da definição */

    struct Aluno Jose; /* Declara uma variável do tipo struct Aluno */

    Jose.nmat   = 456;
    Jose.nota[0]= 7.5;
    Jose.nota[1]= 5.2;
    Jose.nota[2]= 8.4;
    Jose.media  = (Jose.nota[0] + Jose.nota[1] + Jose.nota[2])/3.0;

    printf("Matrícula: %d\n", Jose.nmat);
    printf("Média    : %.2f\n", Jose.media);

    system("PAUSE");
    return 0;
}
```

Declarando uma variável do tipo definido

- A instrução:

```
struct Aluno Jose;
```

- Declara uma variável de nome Jose do tipo struct Aluno.
- Essa declaração reserva espaço de memória suficiente para armazenar todos os membros da estrutura:
  - 4 bytes para nmat;
  - 12 bytes para a matriz nota;
  - 4 bytes para média;

Declarando uma variável do tipo definido

- Os membros de uma variável estrutura são armazenados em sequência contínua de memória.

Roteiro

- Criando novos tipos de dados com struct
- Novos nomes para os tipos existentes
- Usando typedef com struct
- Inicializando estruturas
- Atribuições entre estruturas
- Estruturas aninhadas
- Passando estruturas para funções
- Funções que retornam uma estrutura
- Matrizes de estruturas
- Estruturas com campos bit

Novos Nomes para os Tipos Existentes: typedef

- Declarações com typedef não produzem novos tipos de dados.
- Criam apenas novos nomes (sinônimos) para os tipos existentes.
- Sintaxe:

```
typedef tipo-existente sinonimo;
```

- Observe o fragmento de código a seguir:

Novos Nomes para os Tipos Existentes: typedef

```c
typedef unsigned char BYTE; /* Cria o sinonimo BYTE */
typedef unsigned int uint; /* Cria o sinonimo uint */

int main()
{
    BYTE ch; /* Declara uma variável do tipo BYTE */
    uint x; /* Declara uma variável do tipo uint */
    unsigned char chmm; /* Os tipos originais podem ser usados */
```

Roteiro

- Criando novos tipos de dados com struct
- Novos nomes para os tipos existentes
- Usando typedef com struct
- Inicializando estruturas
- Atribuições entre estruturas
- Estruturas aninhadas
- Passando estruturas para funções
- Funções que retornam uma estrutura
- Matrizes de estruturas
- Estruturas com campos bit

Usando typedef com struct

- Com estruturas, podemos utilizar typedef de três formas:

```
struct Aluno
{
    int nmat;
    float nota[3];
    float media;
};

typedef struct Aluno Aluno;

Aluno Jose;
```

Usando typedef com struct

- Outro modo é criar um sinônimo na mesma instrução de definição da estrutura:

```
typedef struct Aluno
{
    int nmat;
    float nota[3];
    float media;
} Aluno;

Aluno Jose;
```

Usando typedef com struct

- E podemos ainda não informar a etiqueta da estrutura:

```
typedef struct
{
    int nmat;
    float nota[3];
    float media;
} Aluno;

Aluno Jose;
```

Usando typedef com struct

- Observe a similaridade entre a declaração de variáveis de tipos simples e de variáveis de tipos definidos pelo usuário:

```
Aluno Jose;
int val;
```

Acessando os Membros da Estrutura

- Uma vez criada a variável estrutura, seus membros podem ser acessados por meio do operador ponto.
- A instrução:

```
Jose.nmat    = 456;
```

- Atribui o valor 456 ao membro nmat da variável José.
- O operador ponto conecta o nome de uma variável estrutura a um membro dela.

Acessando os Membros da Estrutura

- A linguagem C trata os membros de uma estrutua como quaisquer outras variáveis simples.
- Por exemplo, Jose.nmat é o nome de uma variável do tipo int e pode ser utilizada em todo lugar em que se use variáveis desse tipo.

Combinando Declarações

- No exemplo anterior, definimos a estrutura e declaramos a variável em duas instruções distintas.
- Essas duas instruções podem ser combinadas em uma.

```
struct Aluno          /*Inicio da definição da estrutura */
{
    int nmat;         /* Número da matrícula */
    float nota[3];    /* Notas */
    float media;      /* Média */
} Jose;               /* Cria a variável José do tipo Aluno */
```

Combinando Declarações

- Podemos declarar quantas variáveis quisermos:

```
struct Aluno          /*Inicio da definição da estrutura */
{
     int nmat;        /* Número da matrícula */
     float nota[3];   /* Notas */
     float media;     /* Média */
} Jose, Ana, Joao;    /* Cria variaveis do tipo Aluno */
```

Combinando Declarações

- A etiqueta pode ser suprimida:

```
struct      /*Inicio da definição da estrutura */
{
    int nmat;          /* Número da matrícula */
    float nota[3];     /* Notas */
    float media;       /* Média */
} Jose, Ana, Joao;     /* Cria variaveis do tipo Aluno */
```

- Observe que a etiqueta, na definição da estrutura, pode ser suprimida caso nenhuma outra variável venha a ser declarada mais adiante.
- Separadas por vírgulas, diversas variáveis podem ser declaradas de uma única vez.

Combinando Declarações

- A combinação da definição da estrutura com a declaração das variáveis torna a escrita mais compacta, entretanto menos clara e menos flexível.

Roteiro

- Criando novos tipos de dados com struct
- Novos nomes para os tipos existentes
- Usando typedef com struct
- Inicializando estruturas
- Atribuições entre estruturas
- Estruturas aninhadas
- Passando estruturas para funções
- Funções que retornam uma estrutura
- Matrizes de estruturas
- Estruturas com campos bit

Inicializando Estruturas

- A inicialização de estruturas é semelhante à inicialização de uma matriz.
- Uma variável do tipo estrutura só pode ser inicializada se criada em tempo de compilação; assim, deve ser das classes extern ou static.

Inicializando Estruturas

- Veja um exemplo:

```
typedef struct
{
    int dia;
    char mes[10];
    int ano;
} Data;

static Data natal = {125, "dezembro", 2001};
static Data aniversario = {30, "julho", 1995};
```

- Se as variáveis inicializadas não forem declaradas como static, a maioria dos compiladores não apresentará nenhum erro e criará as variáveis desse tipo automaticamente.

Inicializando Estruturas

- As variáveis natal e aniversário são inicializadas na mesma instrução de suas declarações.
- Os valores a serem atribuídos a seus membros devem ser colocados na ordem em que foram definidos na estrutura, separados por vírgulas e entre chaves.

Definindo e Inicializando na mesma Instrução

- No exemplo anterior, definimos a estrutura e declaramos a estrutura e declaramos a variável inicializadas em duas instruções distintas.
- Quando não utilizamos a palavra typedef essas duas instruções podem ser combinadas em uma.

Definindo e Inicializando na mesma Instrução

```
struct Data
{
    int dia;
    char mes[10];
    int ano;
} natal = {25,"dezembro", 2001},
  aniversario = {30, "julho", 1995};
```

Roteiro

- Criando novos tipos de dados com struct
- Novos nomes para os tipos existentes
- Usando typedef com struct
- Inicializando estruturas
- Atribuições entre estruturas
- Estruturas aninhadas
- Passando estruturas para funções
- Funções que retornam uma estrutura
- Matrizes de estruturas
- Estruturas com campos bit

Atribuições entre Estruturas

- Uma variável estrutura pode ser atribuída a outra do mesmo tipo por meio de uma atribuição simples:

```
static Data aniversario = {30, "julho", 2008};
Data Andre;

Andre = aniversario;
```

Atribuições entre Estruturas

- Certamente, constatamos que essa é uma estupenda capacidade quando pensamos a respeito dela:
  - Todos os valores dos membros da estrutura estão realmente sendo atribuídos de uma única vez aos correspondentes da outra.
- Uma expressão de atribuição tão simples não poderia ser usada para matrizes, que devem ser atribuídas elemento a elemento.

Atribuições entre Estruturas

- Note que a atribuição só é permitida entre estruturas do mesmo tipo.
- Se for atribuída uma variável de um tipo a outra de outro tipo, o compilador acusará um erro.

Operações entre Estruturas

- Para exemplificar o modo de executar operações entre estruturas, vamos montar um tipo de dado que armazene o número de peças vendidas por uma loja num determinado dia e o valor total de venda.
- Em seguida, criaremos duas variáveis com os dados de dois dias e uma terceira variável para armazenar o total de vendas.

Operações entre Estruturas

```
typedef struct
{
    int pecas;
    float preco;
} Venda;

Venda A = {20, 110.0}, B = {3, 16.5}, Total;
```

- Você poderia pensar em adicionar os valores das variáveis A e B por meio de uma instrução simples, como:

```
Total = A + B; /* ERRADO */
```

Operações entre Estruturas

- Em linguagem C, operações simples como a soma não estão definidas para tipos criados com a palavra struct.
- A soma deve ser efetuada membro a membro:

```
Total.pecas = A.pecas + B.pecas;
Total.preco = A.preco + B.preco;
```

Roteiro

- Criando novos tipos de dados com struct
- Novos nomes para os tipos existentes
- Usando typedef com struct
- Inicializando estruturas
- Atribuições entre estruturas
- Estruturas aninhadas
- Passando estruturas para funções
- Funções que retornam uma estrutura
- Matrizes de estruturas
- Estruturas com campos bit

Estruturas Aninhadas

- Exatamente como podemos ter matrizes em que cada elemento é outra matriz, podemos definir estruturas como membros que sejam outras estruturas.

Prog3.c

```c
/* Prog3.c */
/* Mostra estruturas aninhadas */
#include <stdio.h>
#include <stdlib.h>

typedef struct
{
    int dia;
    char mes[10];
    int ano;
} Data;
```

Estruturas Aninhadas

```
13  typedef struct
14  {
15      int pecas;
16      float preco;
17      Data diavenda;
18  } Venda;
19
20  int main()
21  {
22      static Venda A = { 20, 110.0, {7,"Novembro",2001} };
23
24      printf("Pecas: %d\n", A.pecas);
25      printf("Preco: %d\n", A.preco);
26      printf("Data : %d de %s de %d\n",
27              A.diavenda.dia, A.diavenda.mes, A.diavenda.ano);
28      system("PAUSE");
29      return 0;
30  }
```

Estruturas Aninhadas

- Nesse exemplo, adicionamos um membro do tipo Data à estrutura Venda, e criamos a variável A do tipo Venda.
- A variável A tem três membros, dos quais um é estrutura do tipo Data.
- Portanto, A.diavenda é o nome de uma variável do tipo Data, e para acessar seus membros devemos aplicar o operador ponto outra vez.

Estruturas Aninhadas

- Logicamente, esse processo não para nesse nível; podemos ter uma estrutura dentro de outra quantas vezes desejarmos.
- Inicializando estruturas aninhadas:
  - A variável A foi inicializada na instrução:

```
static Venda A = { 20, 110.0, {7,"Novembro",2001} };
```

  - Observe que o membro diavenda, por ser uma estrutura, é inicializado com seus membros entre chaves e separados por vírgulas.

Roteiro

- Criando novos tipos de dados com struct
- Novos nomes para os tipos existentes
- Usando typedef com struct
- Inicializando estruturas
- Atribuições entre estruturas
- Estruturas aninhadas
- Passando estruturas para funções
- Funções que retornam uma estrutura
- Matrizes de estruturas
- Estruturas com campos bit

Passando Estruturas para Funções

- As estruturas podem ser passadas como argumentos de funções da mesma forma que variáveis simples.
- O nome de uma estrutura em C não é um endereço, portanto pode ser acessada por valor.

Roteiro

- Criando novos tipos de dados com struct
- Novos nomes para os tipos existentes
- Usando typedef com struct
- Inicializando estruturas
- Atribuições entre estruturas
- Estruturas aninhadas
- Passando estruturas para funções
- Funções que retornam uma estrutura
- Matrizes de estruturas
- Estruturas com campos bit

Funções que Retornam uma Estrutura

- A linguagem C permite que as funções retornem uma estrutura completa para outra função.
- Vamos exemplificar com uma função que recebe duas estruturas como argumento e retorna uma estrutura com a soma dos argumentos.

Funções que Retornam uma Estrutura

Prog4.c

```
1  /* Prog4.c */
2  /* Mostra passagem de estruturas para funções por valor */
3  /* Mostra uma função que retorna uma estrutura */
4  #include <stdio.h>
5  #include <stdlib.h>
6
7  typedef struct Venda
8  {
9      int pecas;
10     float preco;
11 } Venda;
12
13 Venda TotalVendas( Venda C, Venda D);/* Protótipo */
14
15 int main()
16 {
17     Venda A, B, Total;
18
```

Funções que Retornam uma Estrutura

```
19      printf("Venda A\n=======\n");
20      printf("Insira o número de peças: ");
21      scanf("%d", &A.pecas);
22      printf("Insira o preço: ");
23      scanf("%f", &A.preco);
24
25      printf("\nVenda B\n=======\n");
26      printf("Insira o número de peças: ");
27      scanf("%d", &B.pecas);
28      printf("Insira o preço: ");
29      scanf("%f", &B.preco);
30      Total = TotalVendas( A,B);/* Estruturas como argumento e retorno */
31
32      printf("\n\nVenda Total\n===========");
33      printf("\nTotal de peças: %d." , Total.pecas );
34      printf("\nPreço total   : %.2f\n" , Total.preco );
35
36      system("PAUSE");
37      return 0;
38  }
```

Funções que Retornam uma Estrutura

```
40  Venda TotalVendas(Venda C, Venda D)
41  {
42      Venda T;
43      T.pecas = C.pecas + D.pecas;
44      T.preco = C.preco + D.preco;
45      return T;
46  }
```

Funções que Retornam uma Estrutura

- O protótipo da função TotalVendas() e as instruções de seu corpo tratam as variáveis argumentos como se trata qualquer tipo simples.
- A função cria as novas variáveis C e D para conter cópia dos valores das variáveis A e B de main(), enviadas como argumentos, como faria com qualquer tipo de variável simples passada por valor.

Funções que Retornam uma Estrutura

- Então, as variáveis C e D não são as mesmas das de main(); a função poderia alterá-las sem contudo alterar as originais.

Roteiro

- Criando novos tipos de dados com struct
- Novos nomes para os tipos existentes
- Usando typedef com struct
- Inicializando estruturas
- Atribuições entre estruturas
- Estruturas aninhadas
- Passando estruturas para funções
- Funções que retornam uma estrutura
- Matrizes de estruturas
- Estruturas com campos bit

Matrizes de Estruturas

- Certamente, uma lista de peças e preços é composta por várias vendas (provavelmente mais de duas).
- Cada venda é descrita por uma variável do tipo Venda.
- Para tratar de várias vendas, é perfeitamente correto pensar em uma matriz de estruturas.

Matrizes de Estruturas

- O programa seguinte, além de realizar nosso objetivo de criar uma matriz de estruturas, em que cada elemento da matriz representa uma venda, proporcionará uma interface simples com o usuário, constituida da escolha entre duas opções assinaladas pelas letras “E” e “L”.
- Se o usuário pressionar “E”, o programa permitirá a entrada das informações de uma venda.

Matrizes de Estruturas

- Caso pressione “L”, o programa listará todas as vendas e imprimirá os totais.
- Vamos ainda aumentar as informações de cada venda, acrescentando a data da venda.
- Usaremos duas novas funções de biblioteca C, atoi() e atof().

Matrizes de Estruturas

- A primeira recebe o endereço de uma variável char (string) como argumento e a converte em um número inteiro correspondente a cadeia recebida.
- Por exemplo, se enviarmos “1234” ela retorna o número inteiro 1234.
- A segunda faz a mesma coisa, mas com variáveis float.

Matrizes de Estruturas

Prog5.c

```
1  /* Prog5.c */
2  /* Mostra uma matriz de estruturas */
3  #include <stdio.h>  /* para gets(), printf(), scanf() */
4  #include <stdlib.h> /* Para system(), atoi(), atof(), exit() */
5  #include <conio.h>  /* Para getch() */
6
7  #define ESC 27
8  #define TRUE 1
9
10 typedef struct
11 {
12     int dia;
13     char mes[10];
14     int ano;
15 } Data;
```

Matrizes de Estruturas

```
17  typedef struct
18  {
19      Data diavenda;
20      int pecas;
21      float preco;
22  } Venda;
23
24  void listavenda(void); /* Protótipo */
25  void novavenda(void);  /* Protótipo */
26
27  Venda vendas[50]; /* Cria matriz de estruturas */
28  Venda Total= {{0,"",0},0,0.0}; /* Cria variável do tipo Venda */
29
30
31  int n=0;
```

Matrizes de Estruturas

Prog5.c

```
33  int main()
34  {
35      char ch;
36      while(TRUE)
37      {
38          printf("Digite E para entrar uma venda\n");
39          printf("       L para listar as vendas\n");
40          printf("       ESC para terminar\n");
41
42          ch=getche();
43          printf("\n");
44          switch(ch)
45          {
46          case 'E':
47              novavenda();
48              break;
49          case 'L':
50              listavenda();
51              break;
52          case ESC: /* ESC não pode ser declarado com const */
53              exit(0);
54          default :
55              printf("Opcao invalida!!\n");
56          }
57
58      }
59      system("PAUSE");
60      return 0;
61  }
```

Matrizes de Estruturas

Prog5.c

```
62  void novavenda()
63  {
64      char temp[80];
65      printf("Dia: ");
66      gets(temp);
67      vendas[n].diavenda.dia=atoi(temp);
68      printf("Mes: ");
69      gets(vendas[n].diavenda.mes);
70      printf("Ano: ");
71      gets(temp);
72      vendas[n].diavenda.ano=atoi(temp);
73      printf("Pecas: ");
74      gets(temp);
75      vendas[n].pecas=atoi(temp);
76      printf("Preco: ");
77      gets(temp);
78      vendas[n].preco=atof(temp);
79      Total.pecas += vendas[n].pecas;
80      Total.preco += vendas[n++].preco;
81  }
```

Matrizes de Estruturas

```
 83  void listavenda()
 84  {
 85      int i;
 86      if(!n)
 87      {
 88          printf("Lista vazia.\n");
 89          return;
 90      }
 91
 92      printf("\n\nRelatório\n");
 93      for (i=0; i<n; i++)
 94      {
 95          printf("%2d de %10s de %4d", vendas[i].diavenda.dia,
 96                  vendas[i].diavenda.mes, vendas[i].diavenda.ano);
 97          printf("%10d", vendas[i].pecas);
 98          printf("%20.2f\n", vendas[i].preco);
 99      }
100      printf("\nTotal");
101      printf("%29d", Total.pecas);
102      printf("%20.2f\n\n", Total.preco);
103  }
```

Declarando a Matriz de Estruturas

- O processo de declaração de uma matriz de estruturas é perfeitamente análogo ao de declaração de qualquer outro tipo de matriz.
- A instrução:

```
Venda vendas[50];
```

- Declara vendas como sendo uma matriz de 50 elementos.
- Cada elemento da matriz é uma estrutura do tipo Venda.

Declarando a Matriz de Estruturas

- Então, vendas[0] é a primeira estrutura do tipo Venda, vendas[1] é a segunda estrutura do tipo Venda, e assim por diante.
- O nome vendas é o de uma matriz (portanto um endereço) em que os elementos são estruturas.
- Observe que, a partir dessa instrução, o compilador providencia espaço contínuo de memória para armazenar 50 estruturas do tipo Venda.

Declarando a Matriz de Estruturas

- Para simplificar, declaramos a matriz como externa, assim todas as funções do programa poderão acessá-la.

Acessando Membros da Matriz de Estruturas

- Membros individuais de cada estrutura são acessados aplicando-se o operador ponto seguido do nome da variável, que, nesse caso, é um elemento da matriz.

```
vendas[n].preco
```

- Observe que o subscrito da matriz é associado a vendas e não ao membro.
- A expressão acima refere-se ao membro preco da n-ésima estrutura da matriz.

Acessando Membros da Matriz de Estruturas

- O esquema global desse programa pode ser aplicado a uma grande variedade de situações, como o controle de um estoque, em que cada variável poderá conter dados sobre um item do estoque, como número do material, preço, tipo, número de peças do estoque, etc.

Roteiro

- Criando novos tipos de dados com struct
- Novos nomes para os tipos existentes
- Usando typedef com struct
- Inicializando estruturas
- Atribuições entre estruturas
- Estruturas aninhadas
- Passando estruturas para funções
- Funções que retornam uma estrutura
- Matrizes de estruturas
- Estruturas com campos bit

Estruturas com Campo bit

- Com a palavra struct podemos definir tipos de dados para manipulação de bits.
- O conjunto de membros consiste nos bits de um char, short ou long.
- Cada membro é formado por um ou mais bits inteiro.
- A estrutura nomeia os membros e determina seus tamanhos.

Estruturas com Campo bit

- Um caractere é impresso na memória de vídeo da console, ocupando dois bytes.
- O primeiro byte é para o seu código ASCII, e o segundo, para um atributo (cor, piscante, reverso, etc.).
- Como exemplo, vamos apresentar um tipo de dado que possa armazenar o caractere e seu atributo:

Estruturas com Campo bit

```c
typedef struct
{
    unsigned short CodAscii: 8; /* Bits 0 a 7 */
    unsigned short CorFrente: 3; /* Bits 8 a 10 */
    unsigned short Intenso: 1; /* Bits 11 */
    unsigned short CorFundo: 3; /* Bits 12 a 14 */
    unsigned short Piscante: 1; /* Bits 15 */
} Int16Bits;

int main()
{
    Int16Bits A;
    A.CodAscii = 'A';
    A.CorFrente = 3;     /* Branco para a cor de frente */
    A.Intenso = 1;       /* Intensificada a cor de frente */
    A.CorFundo = 0;      /* Preto para a cor de fundo */
    A.Piscante = 1;      /* Escolhe atributos piscante */
}
```

Referências

- MIZRAHI, Victorine V. **Treinamento em Linguagem C**, São Paulo: Pearson Prentice Hall, 2008;